EVM Dashboard — Validation Report

CE 725 Construction Economics and Finance · Course project, Topic 7

Status of this report. Part A (synthetic benchmark) results come from runs on 21-Oct-2026; Part B (real-data benchmark, Section 6B), the defects D7–D16 and the updated test counts come from runs on 29-Oct-2026 (evidence: `reports/test_results.txt`, `reports/benchmark_comparison.md`, `reports/manual_check_comparison.md`, `reports/browser_test_log.txt`, `reports/browser_smart_reader_log.txt`). The independent benchmark calculations were prepared with AI assistance (Claude) separately from the dashboard code. **The course requires the team to verify every number against a manual benchmark; the Validation Engineer must redo Section 4 on paper and sign Section 15 before submission.** Until then, Section 4 is “prepared, awaiting team verification”.

1. Validation objective

To show that every metric the dashboard reports — PV, EV, AC, CV, SV, CPI, SPI, EAC (methods A–C), ETC, VAC, forecast overrun, TCPI to BAC and TCPI to EAC — equals an independently prepared manual result for two reporting periods (course requirement, Topic 7), and that undefined, invalid and edge-case inputs are handled without displaying fabricated numbers.

2. Benchmark project

Synthetic, illustrative G+4 RCC residential block. Four work packages; BAC ₹4,00,00,000 (sum of WP budgets). Two monthly status dates: Period 1 = 31-Jan-2026, Period 2 = 28-Feb-2026. A work-package case was chosen deliberately because it also tests budget weighting, the most common EV error. It also contains cost without progress (WP4, Period 1: ₹1,00,000 spent at 0% complete — a procurement advance), which must not be treated as an error.

3. Input data (cumulative)

Work package	Budget (₹)	P1 planned %	P1 actual %	P1 AC (₹)	P2 planned %	P2 actual %	P2 AC (₹)
WP1 Substructure	80,00,000	100	100	84,00,000	100	100	84,00,000
WP2 RCC superstructure	1,60,00,000	40	30	52,00,000	70	55	98,00,000
WP3 Masonry and finishes	1,20,00,000	0	0	0	10	5	7,00,000
WP4 MEP services	40,00,000	5	0	1,00,000	10	5	3,00,000
Total	**4,00,00,000**						

File: `data/benchmark_case_work_packages.csv`. Assumptions: all values cumulative; actual % is physical progress against budgeted scope; one baseline.

4. Manual calculations (working shown; ₹ lakh = ₹1,00,000)

Period 1 — 31-Jan-2026

- PV = 80×1.00 + 160×0.40 + 120×0 + 40×0.05 = 80 + 64 + 0 + 2 = **146.00 lakh**
- EV = 80×1.00 + 160×0.30 + 120×0 + 40×0 = 80 + 48 = **128.00 lakh**
- AC = 84 + 52 + 0 + 1 = **137.00 lakh**
- CV = 128 − 137 = **−9.00 lakh**; SV = 128 − 146 = **−18.00 lakh**
- CPI = 128/137 = **0.934307**; SPI = 128/146 = **0.876712**
- EAC(A) = BAC/CPI = 400 × 137/128 = **428.125 lakh**; ETC = 428.125 − 137 = 291.125; VAC = −28.125; overrun 28.125/400 = **7.03125%**
- EAC(B) = 137 + (400 − 128)/(CPI×SPI); CPI×SPI = (128×128)/(137×146) = 16,384/20,002 = 0.819118; 272/0.819118 = 272×20,002/16,384 = **332.064453**; EAC(B) = **469.064453 lakh**; ETC 332.064453; VAC −69.064453; overrun **17.266113%**
- EAC(C) = 137 + 272 = **409.00 lakh**; ETC 272; VAC −9; overrun **2.25%**
- TCPI to BAC = (400 − 128)/(400 − 137) = 272/263 = **1.034221**
- TCPI to EAC: A = 272/291.125 = 0.934307 (= CPI ✓); B = 272/332.064453 = 0.819118 (= CPI×SPI ✓); C = 272/272 = 1 ✓

Period 2 — 28-Feb-2026

- PV = 80 + 160×0.70 + 120×0.10 + 40×0.10 = 80 + 112 + 12 + 4 = **208.00 lakh**
- EV = 80 + 160×0.55 + 120×0.05 + 40×0.05 = 80 + 88 + 6 + 2 = **176.00 lakh**
- AC = 84 + 98 + 7 + 3 = **192.00 lakh**
- CV = **−16.00 lakh**; SV = **−32.00 lakh**; CPI = 176/192 = **0.916667**; SPI = 176/208 = **0.846154**
- EAC(A) = 400 × 192/176 = **436.363636 lakh**; ETC 244.363636; VAC −36.363636; overrun **9.090909%**
- EAC(B): CPI×SPI = 30,976/39,936 = 0.775641; 224/0.775641 = 224×39,936/30,976 = **288.793388**; EAC(B) = **480.793388 lakh**; overrun **20.198347%**
- EAC(C) = 192 + 224 = **416.00 lakh**; overrun **4.00%**
- TCPI to BAC = 224/208 = **1.076923**

Weighting check. An unweighted average of Period 2 actual percentages, (100+55+5+5)/4 = 41.25%, would give EV = 165 lakh — ₹11 lakh (6.3%) below the correct budget-weighted 176 lakh. The dashboard must give 176.

Independent cross-checks of the paper working. (a) `benchmark/independent_benchmark_calc.py` recomputes everything with exact fractions and imports no dashboard code (enforced by a test that inspects its import statements); (b) `benchmark/benchmark_workbook.xlsx` computes the same with plain spreadsheet formulas (73 formulas, recalculated in LibreOffice, 0 errors). All three agree to the last digit shown above.

5. Dashboard calculation procedure

The dashboard values were produced through the same path a user follows: read the CSV → map columns → validate → derive PV/EV/AC → calculate (`tests/test_benchmark.py`, `benchmark/compare_with_dashboard.py`). The same project was also entered (i) as pre-

calculated PV/EV/AC and (ii) as project-level percentages (36.5%/32% and 52%/44%), to check the three input modes agree. In the browser, the benchmark PV/EV/AC were entered through the manual-entry table and loaded; the Overview showed CPI 0.917, SPI 0.846, EAC(A) ₹436.36 lakh and TCPI 1.077 for Period 2, matching Section 4.

6. Comparison

Tolerances: money ±₹0.01 (one paisa); indices $\pm 1\times10^{-9}$; percentages $\pm 1\times10^{-7}$. Values were compared at full precision — no rounding before comparison.

- **work_package** input mode: 54 values compared, 54 PASS, 0 FAIL; max |difference| = 7.45e-09
- **precalc** input mode: 54 values compared, 54 PASS, 0 FAIL; max |difference| = 7.45e-09
- **progress** input mode: 54 values compared, 54 PASS, 0 FAIL; max |difference| = 7.45e-09

Full table (work-package input mode; the other two modes give identical results):

Metric	Method	Period	Manual result	Dashboard result	Difference	Tolerance	Status
BAC		2026-01-31	40,000,000.0000	40,000,000.0000	0.00e+00	0.01	PASS
PV		2026-01-31	14,600,000.0000	14,600,000.0000	0.00e+00	0.01	PASS
EV		2026-01-31	12,800,000.0000	12,800,000.0000	0.00e+00	0.01	PASS
AC		2026-01-31	13,700,000.0000	13,700,000.0000	0.00e+00	0.01	PASS
CV		2026-01-31	-900,000.0000	-900,000.0000	0.00e+00	0.01	PASS
SV		2026-01-31	-1,800,000.0000	-1,800,000.0000	0.00e+00	0.01	PASS
CPI		2026-01-31	0.9343065693	0.9343065693	4.31e-11	1e-09	PASS
SPI		2026-01-31	0.8767123288	0.8767123288	-3.29e-11	1e-09	PASS
TCPI_BAC		2026-01-31	1.0342205323	1.0342205323	1.94e-11	1e-09	PASS
EAC	A	2026-01-31	42,812,500.0000	42,812,500.0000	0.00e+00	0.01	PASS
ETC	A	2026-01-31	29,112,500.0000	29,112,500.0000	0.00e+00	0.01	PASS
VAC	A	2026-01-31	-2,812,500.0000	-2,812,500.0000	0.00e+00	0.01	PASS
OVERRUN	A	2026-01-31	2,812,500.0000	2,812,500.0000	0.00e+00	0.01	PASS
OVERRUN_PCT	A	2026-01-31	7.03125000	7.03125000	0.00e+00	1e-07	PASS
TCPI_EAC	A	2026-01-31	0.9343065693	0.9343065693	4.31e-11	1e-09	PASS
EAC	B	2026-01-31	46,906,445.3125	46,906,445.3125	0.00e+00	0.01	PASS
ETC	B	2026-01-	33,206,445.31	33,206,445.31	0.00e+00	0.01	PASS

Metric	Method	Period	Manual result	Dashboard result	Difference	Tolerance	Status
		31	25	25			
VAC	B	2026-01-31	-6,906,445.3125	-6,906,445.3125	0.00e+00	0.01	PASS
OVERRUN	B	2026-01-31	6,906,445.3125	6,906,445.3125	0.00e+00	0.01	PASS
OVERRUN_PCT	B	2026-01-31	17.26611328	17.26611328	5.00e-11	1e-07	PASS
TCPI_EAC	B	2026-01-31	0.8191180882	0.8191180882	-8.82e-12	1e-09	PASS
EAC	C	2026-01-31	40,900,000.0000	40,900,000.0000	0.00e+00	0.01	PASS
ETC	C	2026-01-31	27,200,000.0000	27,200,000.0000	0.00e+00	0.01	PASS
VAC	C	2026-01-31	-900,000.0000	-900,000.0000	0.00e+00	0.01	PASS
OVERRUN	C	2026-01-31	900,000.0000	900,000.0000	0.00e+00	0.01	PASS
OVERRUN_PCT	C	2026-01-31	2.25000000	2.25000000	0.00e+00	1e-07	PASS
TCPI_EAC	C	2026-01-31	1.0000000000	1.0000000000	0.00e+00	1e-09	PASS
BAC		2026-02-28	40,000,000.0000	40,000,000.0000	0.00e+00	0.01	PASS
PV		2026-02-28	20,800,000.0000	20,800,000.0000	0.00e+00	0.01	PASS
EV		2026-02-28	17,600,000.0000	17,600,000.0000	0.00e+00	0.01	PASS
AC		2026-02-28	19,200,000.0000	19,200,000.0000	0.00e+00	0.01	PASS
CV		2026-02-28	-1,600,000.0000	-1,600,000.0000	0.00e+00	0.01	PASS
SV		2026-02-28	-3,200,000.0000	-3,200,000.0000	0.00e+00	0.01	PASS
CPI		2026-02-28	0.9166666667	0.9166666667	-3.33e-11	1e-09	PASS
SPI		2026-02-28	0.8461538462	0.8461538462	-4.62e-11	1e-09	PASS
TCPI_BAC		2026-02-28	1.0769230769	1.0769230769	2.31e-11	1e-09	PASS
EAC	A	2026-02-28	43,636,363.6364	43,636,363.6364	7.45e-09	0.01	PASS
ETC	A	2026-02-28	24,436,363.6364	24,436,363.6364	0.00e+00	0.01	PASS
VAC	A	2026-02-28	-3,636,363.6364	-3,636,363.6364	-4.66e-09	0.01	PASS

Metric	Method	Period	Manual result	Dashboard result	Difference	Tolerance	Status
OVERRUN	A	2026-02-28	3,636,363.6364	3,636,363.6364	4.66e-09	0.01	PASS
OVERRUN_PCT	A	2026-02-28	9.09090909	9.09090909	9.10e-12	1e-07	PASS
TCPI_EAC	A	2026-02-28	0.9166666667	0.9166666667	-3.33e-11	1e-09	PASS
EAC	B	2026-02-28	48,079,338.8430	48,079,338.8430	7.45e-09	0.01	PASS
ETC	B	2026-02-28	28,879,338.8430	28,879,338.8430	7.45e-09	0.01	PASS
VAC	B	2026-02-28	-8,079,338.8430	-8,079,338.8430	-7.45e-09	0.01	PASS
OVERRUN	B	2026-02-28	8,079,338.8430	8,079,338.8430	7.45e-09	0.01	PASS
OVERRUN_PCT	B	2026-02-28	20.19834711	20.19834711	3.80e-11	1e-07	PASS
TCPI_EAC	B	2026-02-28	0.7756410256	0.7756410256	4.10e-11	1e-09	PASS
EAC	C	2026-02-28	41,600,000.0000	41,600,000.0000	0.00e+00	0.01	PASS
ETC	C	2026-02-28	22,400,000.0000	22,400,000.0000	0.00e+00	0.01	PASS
VAC	C	2026-02-28	-1,600,000.0000	-1,600,000.0000	0.00e+00	0.01	PASS
OVERRUN	C	2026-02-28	1,600,000.0000	1,600,000.0000	0.00e+00	0.01	PASS
OVERRUN_PCT	C	2026-02-28	4.00000000	4.00000000	0.00e+00	1e-07	PASS
TCPI_EAC	C	2026-02-28	1.0000000000	1.0000000000	0.00e+00	1e-09	PASS

6B. Part B — real-data benchmark: the manual check case

Why a second benchmark. The Part A benchmark is synthetic. Part B repeats the validation on a **real** project, small enough to work by hand, and adds something a synthetic case cannot: the real final cost is known, so the forecasts themselves can be checked.

Project. IJzertoren Memorial Square, Diksmuide, Belgium (civil works, 2013–14), project C2015-33 of the DSLIB empirical database, Ghent University (Batselier & Vanhoucke, 2015). Its 12 activities, recorded in the project-control software ProTrack, were grouped into 4 work packages by trade (`tools/build_manual_check_case.py`): WP budget = sum of activity budgets; planned % = sum of activity PV ÷ WP budget; actual % = sum of activity EV ÷ WP budget; AC = sum of activity AC. Percentages were rounded to 2 decimals so the case can be worked with a calculator. BAC €214,417.71; baseline 14-Oct-2013 to 20-Dec-2013; five month-end reports; the project finished on 28-Feb-2014 at €224,789.67 (4.8 % over budget, about 2 months late).

Agreement with the recorded values. The 4-package file reproduces ProTrack's own project PV, EV and AC within €3.79 (largest difference, caused only by the 2-decimal rounding; proven bound 0.005 % × BAC

= €10.72), and its CPI and SPI to within 0.0005, for all five reports (`tests/test_manual_check_case.py::test_reproduces_protrack_recorded_values`).

Inputs for the two benchmark periods (from `data/real/C2015-33_ijzertoren_manual_check_work_packages.csv`):

Work package	Budget (€)	30-Nov planned %	30-Nov actual %	30-Nov AC (€)	31-Dec planned %	31-Dec actual %	31-Dec AC (€)
WP1 Demolition and sewerage	49,231.67	100.00	92.31	38,459.62	100.00	100.00	39,821.67
WP2 Earthworks and foundations	58,025.66	100.00	32.64	50,759.94	100.00	58.03	69,412.98
WP3 Kerbs, paving and concrete	78,356.70	5.75	12.40	10,271.32	100.00	22.27	40,197.69
WP4 Finishing (street furniture, signage)	28,803.68	0.00	19.74	6,201.39	100.00	39.49	7,162.11
Total (BAC)	**214,417.71**						

Manual working — 30-Nov-2013 (period 2)

- PV = 49,231.67 + 58,025.66 + 78,356.70×0.0575 + 0 = 49,231.67 + 58,025.66 + 4,505.51 = **€111,762.84**
- EV = 49,231.67×0.9231 + 58,025.66×0.3264 + 78,356.70×0.1240 + 28,803.68×0.1974 = 45,445.75 + 18,939.58 + 9,716.23 + 5,685.85 = **€79,787.41**
- AC = 38,459.62 + 50,759.94 + 10,271.32 + 6,201.39 = **€105,692.27**
- CV = 79,787.41 − 105,692.27 = **−€25,904.86**; SV = 79,787.41 − 111,762.84 = **−€31,975.43**
- CPI = 79,787.41 / 105,692.27 = **0.754903**; SPI = 79,787.41 / 111,762.84 = **0.713899**
- EAC A = 214,417.71 / 0.754903 = **€284,033.47**; EAC B = 105,692.27 + 134,630.30 / (0.754903 × 0.713899) = **€355,505.12**; EAC C = 105,692.27 + 134,630.30 = **€240,322.57**
- ETC A = 284,033.47 − 105,692.27 = €178,341.20; VAC A = −€69,615.76; forecast overrun A = **32.47 %**
- TCPI to BAC = (214,417.71 − 79,787.41) / (214,417.71 − 105,692.27) = 134,630.30 / 108,725.44 = **1.238259**
- Work-package view: WP4 has value earned before any was planned (PV = 0), so its SPI is **undefined** and its status is *Insufficient data*; WP2 CPI = 18,939.58 / 50,759.94 = **0.373** (Critical).

Manual working — 31-Dec-2013 (period 3, about half complete)

- PV = BAC = **€214,417.71** (the baseline finish, 20-Dec-2013, has passed)
- EV = 49,231.67 + 58,025.66×0.5803 + 78,356.70×0.2227 + 28,803.68×0.3949 = 49,231.67 + 33,672.29 + 17,450.04 + 11,374.57 = **€111,728.57**
- AC = 39,821.67 + 69,412.98 + 40,197.69 + 7,162.11 = **€156,594.45**
- CV = **−€44,865.88**; SV = **−€102,689.14**; CPI = **0.713490**; SPI = **0.521079**; % complete = 52.11 %

- EAC A = **€300,519.58**; EAC B = **€432,800.41**; EAC C = **€259,283.59**; TCPI to BAC = 102,689.14 / 57,823.26 = **1.775914**
- Identity checks: TCPI to EAC A = CPI = 0.713490 ✓; TCPI to EAC C = 1 ✓.

Comparison. `benchmark/compare_manual_check_case.py` compares the hand-calculation workbook (`data/manual_check/IJzertoren_manual_check_workbook.xlsx`, Excel formulas only, 502 formulas, 0 errors) with the dashboard for **all five reports**: **90 values compared, 90 PASS, 0 FAIL** (money ±€0.01, indices $\pm 10^{-9}$; full table in `reports/manual_check_comparison.md`). The automated test also compares every work package's PV, EV, AC, CV, SV, CPI, SPI and status at every date. On screen, the dashboard shows CPI 0.755 / SPI 0.714 / EAC (A) €284,033 for 30-Nov (interface test `test_manual_check_case_at_the_two_benchmark_dates`) and CPI 0.713 / SPI 0.521 / EAC (A) €300,520 for 31-Dec (interface test and browser screenshot `reports/screenshots/m_overview.png`), matching the working above.

Forecast check (only possible with real data). Each forecast compared with the real final cost of €224,789.67:

Report	% complete	EAC A	EAC B	EAC C	Error A	Error B	Error C
31-Oct-2013	15.5 %	188,834	322,625	210,445	−16.0 %	+43.5 %	−6.4 %
30-Nov-2013	37.2 %	284,033	355,505	240,323	+26.4 %	+58.2 %	+6.9 %
31-Dec-2013	52.1 %	300,520	432,800	259,284	+33.7 %	+92.5 %	+15.3 %
31-Jan-2014	88.9 %	219,115	222,161	218,593	−2.5 %	−1.2 %	−2.8 %

What this shows. The formulas are correct (Part A and Part B), yet the forecasts are poor until late in this project, because most of the overspend happened early and the project then recovered (CPI rose from 0.71 to 0.98). Method B, which assumes schedule pressure costs extra, was the most pessimistic. This supports the dashboard's design choice to show all methods side by side, to label every EAC as an assumption-based forecast, and not to present any one method as "the" answer. A second real project, the apartment building C2014-05 cut at 50 % complete (`data/real/forecast_test/`; the 50 % report is interpolated between two real reports and labelled as such), shows the same ordering: the dashboard reproduces the answer key exactly, and the forecast errors are A +7.0 %, B +32.5 %, C −1.5 % (`tests/test_forecast_at_50pct.py`).

7. Discrepancies and defects found

No numerical discrepancy was found between the manual benchmark and the dashboard (maximum |difference| = $₹7.5\times10^{-9}$, floating-point representation). Because a first-run match does not prove the tests can detect errors, mutation testing was performed (Section 12). The following genuine defects and findings arose during development and testing:

#	Found by	Defect / finding	Root cause	Correction	Retest
D1	Unit test run	Test "expected file independent of src" failed although the benchmark script imports nothing from the app	Test searched the text for "from src" and matched a phrase in the script's docstring (defect in the test, not the app)	Test rewritten to inspect actual import statements (AST)	Pass
D2	Browser test	After a rejected upload, the next (valid) file loaded under the project	Settings widgets kept their state across files (same widget	Widgets keyed to the specific file and mode	Pass: second upload took the file's own name

#	Found by	Defect / finding	Root cause	Correction	Retest
		name typed for the rejected file, not the name in the new file	keys)		
D3	Browser test (screenshot review)	A non-numeric AC produced two errors for one cell (“not a number” and “missing”)	Missing-value check re-flagged a cell already rejected by parsing	Missing check skips cells already in error	Pass: 5 errors instead of 6
D4	Screenshot review	A warning named the internal field `planned_pct` instead of the user’s column `planned_progress_pct`	Internal column name passed as field label	Mapped to user-facing names	Pass
D5	Data generation	Progress-based demo file EV for 31-Aug-2026 = ₹7,02,50,000.40 vs ₹7,02,50,000 from WP data	Percentages stored to 6 decimals	Not a code defect; documented as limitation (rounded % inputs lose precision)	Recorded
D6	Screenshot review	Signboard clipped, KPI labels truncated, BAC/EAC chart labels overlapping, CPI/SPI axis cutting off the 1.00 line, preview dates showing “00:00:00”	Layout defaults	Labels shortened, annotations repositioned, axis range includes 1.00 and the critical line, dates formatted	Pass (screenshots in `reports/screenshots/`)
D7	Team’s own site sheet (`tests/fixtures/site_status_sheet.xlsx`)	Upload unusable: header on row 4 read as data, status date above the table ignored, 0.75 read as 0.75 %, “Project totals” row read as an activity	Reader assumed a template-shaped file	Smart reader (`src/smart_reader.py`): header detection, status date from the cell above, fraction and lakh/crore conversion, summary-row removal — every change listed on screen	Pass: 38 tests; BAC and PV equal the sheet’s own totals
D8	Unit test	Validation messages numbered rows 1, 2, 3… instead of the real spreadsheet rows	Row numbers assigned after cleaning	Smart reader passes original row numbers to validation	Pass: A03 reported on row 7
D9	Team upload (Prashant’s sheet)	Supplied PV/EV columns flagged “not needed” plus a validation warning, which looked like an error	Unused columns reported, not checked	Supplied PV/EV cross-checked row by row against budget × progress %; differences named per	Pass: all 12 match; mismatch test

#	Found by	Defect / finding	Root cause	Correction activity	Retest
D10	Team screenshot	One-date file: charts nearly empty, same date repeated six times on the x-axis	Automatic date ticks on a single point	One label per reporting date; explicit “only one reporting date” note	Pass: `tests/test_charts.py`
D11	Team usability feedback	Home button, reporting date and forecast method hidden in the collapsible sidebar	Layout	Always-visible top bar; sidebar keeps display settings	Pass: app tests
D12	App test (AI error)	The AI’s summary of the manual check case gave CPI at 31-Dec-2013 as 0.714; correct value 0.71349 → 0.713	Rounding slip in the AI’s prose, not in the code	Test expectation and documents corrected	Pass
D13	Test design	First ProTrack-agreement test used a guessed €3 tolerance; observed difference €3.79	Tolerance not derived	Replaced by the proven rounding bound (0.005 % × BAC = €10.72)	Pass
D14	Screenshot automation	A styling rule targeted a page element that does not exist in Streamlit 1.66, so the panel had no card styling	Element name from an older version	Panel given an explicit key and styled by it	Pass (screenshots)
D15	Browser automation	Values typed into the entry table at machine speed were partly lost	Test typed while the page was still updating	Test types at human pace (waits after each entry); both input paths then pass	Pass: 13/13 checks (`reports/browser_smart_reader_log.txt`)
D16	Screenshot review (one-date upload)	The “only one reporting date” note, drawn as a box inside the plot, covered the BAC, EAC, status-date and “1.00 = on plan” labels	Note placed inside the plot area (D10 fix)	Note moved under the date axis; test now requires it there and not inside the plot	Pass: `tests/test_charts.py`; `reports/screenshots/m_one_date.png`

8–10. Root causes, corrections and retest

Given per defect in Section 7. After all corrections the complete suite was rerun on 29-Oct-2026: **164 passed, 0 failed** (`reports/test_results.txt`), and both browser runs were repeated (`reports/browser_test_log.txt`, `reports/browser_smart_reader_log.txt`).

11. Edge-case and workflow testing

Case	Expected behaviour	Evidence	Result
AC = 0	CPI undefined (not 0), reason shown; CV defined	`test_cpi_undefined_when_ac_zero`	Pass
PV = 0	SPI undefined	`test_spi_undefined_when_pv_zero`	Pass
EV = 0	CPI = 0; EAC A and B undefined; C defined	`test_zero_earned_value_…`	Pass
AC = BAC with work remaining	TCPI to BAC undefined (“no longer possible”)	`test_tcpi_bac_undefined_when_ac_equals_bac`	Pass
AC > BAC	TCPI to BAC undefined, never negative	`test_tcpi_bac_undefined_when_budget_exceeded`	Pass
EAC below / above BAC	VAC positive / negative	`test_eac_below_bac…`, `test_eac_above_bac`	Pass
Project complete (EV = PV = BAC)	SPI = 1, ETC = 0, TCPI N/A	`test_project_completion`	Pass
Missing reporting period	Warning; chart line broken	`test_missing_monthly_period…`; `with_gap_breaks`	Pass
Negative cumulative AC / BAC	Blocking error	`test_negative_cumulative_ac_is_error`, `test_missing_project_name_and_bad_bac`	Pass
Progress < 0 or > 100	Blocking error	`test_progress_out_of_range`	Pass
Decreasing cumulative values	Warning, value unchanged	`test_decreasing_cumulative_progress…`	Pass
Cost with no new progress	No error	`test_cost_without_progress_is_not_an_error`	Pass
Cumulative and period AC inconsistent	Warning; cumulative used, never summed	`test_cumulative_and_period_ac_inconsistent`	Pass
Missing value	Error, never zero	`test_missing_value_is_error_not_zero`	Pass
Unweighted WP average	Budget-weighted EV used	`test_work_package_ev_is_budget_weighted…`	Pass
All 4 forecast methods, all 9 dates, all units	Render without error; method D “N/A” until ETC entered	`tests/test_app.py`	Pass
Invalid file upload (browser)	Errors listed by row; no load button; current project unchanged	browser log	Pass
Valid CSV upload (browser)	Loads after confirmation; CPI 0.947	browser log	Pass
Manual entry → load (browser)	Benchmark P2 values reproduced	browser log	Pass
Editing a single cell in the manual table (browser)	Edit re-validates	Scripted edit did not register in the canvas grid	**Not verified** — team to check by hand (2 min): change an AC to −5 and confirm a blocking error appears
Upload ↔ manual entry equivalence	Identical results	`test_manual_entry_and_upload_give_identica`	Pass

Case	Expected behaviour	Evidence	Result
		`l_results`	
Older library versions	Suite passes	pandas 2.2.3 / Streamlit 1.50 / Plotly 5.24 venv: 79 passed (21-Oct; not repeated after later changes)	Pass (21-Oct)
Messy site sheet: title rows, status date above table, fractions, totals row, no AC	Understood; AC requested	`tests/test_smart_reader.py`; browser log (two entry paths)	Pass
Odd headers, ₹ signs, “137 lakh”, dd/mm dates, semicolons, Primavera BCWS/BCWP/ACWP, cover sheet	Mapped correctly; unknown columns not guessed	`tests/test_smart_reader.py`	Pass
Supplied PV/EV that disagree with budget × %	Each differing row named; warning kept	`test_supplied_ev_that_disagrees_is_reported_by_activity`	Pass
Single reporting date	Charts labelled once, note shown	`tests/test_charts.py`	Pass
Work package with value earned before it was planned (PV = 0)	SPI undefined, status “Insufficient data”	manual check case, 30-Nov-2013	Pass

12. Mutation testing (can the tests catch typical errors?)

Each error below was injected into a temporary copy of the code; the suite was run; the copy discarded.

Injected error	Tests failing
SPI inverted (PV/EV)	8
CV sign reversed (AC − EV)	6
EAC(B) written as BAC/(CPI×SPI)	5
TCPI to BAC forced to 1	4
EV from an unweighted average of WP percentages	2
Period AC used as if cumulative	1
CPI returned as 0 when AC = 0	2

All seven were detected. Faults injected into later code (27–29 Oct 2026) were also all detected: smart reader 10/10 (e.g. header always row 1, fractions not converted, baseline start accepted as status date, content check skipped), PV/EV cross-check 3/3, home screen and work-package analysis 7/7, charts and top bar 6/6 (e.g. dates not labelled, amber zone missing, Home only in the sidebar, method B formula changed — caught by the real-data workbook comparison).

13. Remaining limitations of the validation

- Two benchmarks: one synthetic (Part A) and one real project grouped into 4 work packages with 2-decimal percentages (Part B). Practitioner review is still recommended.
- Method D (independent ETC) is checked by unit and interface tests, not by the paper benchmark (it is a user input, not a formula).

- Charts and commentary were checked by inspection and interface tests, not against an independent reference.
- Manual-table cell editing in the browser not verified by automation (Section 11).

14. Conclusion

For the synthetic benchmark, all 54 values per input mode (162 comparisons) match the independently prepared results within one paisa / 10^{-9}, with no discrepancies; undefined and edge cases are reported as undefined or flagged rather than shown as numbers; and the test suite detects all seven typical formula errors injected. For the real-data benchmark, the hand-calculation workbook and the dashboard agree on all 90 values for all five reports, and the data reproduces the values recorded in ProTrack. The real project also shows that correct formulas can still give poor forecasts early in a project, which the dashboard communicates by showing all methods and labelling them as assumption-based. Subject to the team's own paper verification (Section 15), the calculation engine is considered validated for the scope tested.

15. Team verification sign-off

Check	Done by	Date	Signature
Period 1 recalculated on paper without the tool; matches Section 4			
Period 2 recalculated on paper without the tool; matches Section 4			
Part B: 30-Nov-2013 and 31-Dec-2013 recalculated on paper from the input table in Section 6B; matches			
`python -m pytest` run on a team laptop: 164 passed			
Manual-table cell edit check (Section 11) done			
Any discrepancy found by the team recorded here and in the AI usage log			